

최종 보고서

LLM기반텍스트RPG

과목 : 객체지향 프로그래밍

교수 : 조현철

팀 : 달팽이와 거북이

안재영(5702904)

강준혁(5945823)

김준형(5702770)

차서훈(5946221)

제출일 : 2026.06.10

프로젝트 제목 : LLM기반텍스트RPG

팀명 : 달팽이와 거북이

구성원 : 안재영(5702904), 강준혁(5945823), 김준형(5702770), 차서훈(5946221)

소스코드 다운로드 링크 <깃허브>

<https://github.com/chani088x/TextRPG-LLM>

시연 동영상 링크 <유튜브>

<https://www.youtube.com/watch?v=IEr2BilG2js> -> 수정 예정

+ 프로젝트 개요

작품주제 선정 배경

각 팀원들이 하고 싶었던 것들이 전부 달라서 이들을 모두 수용하기 위한 프로젝트를 찾은 결과 LLM기반 텍스트 RPG라는 결과에 도달하게 되었습니다. 중세 판타지로 세계관을 정한 이유는 텍스트 RPG에서 가장 흔하고 유명한 세계관이기 때문에 학습 데이터 측면에서 유리하다고 생각했기 때문입니다.

벤치마킹 자료

<https://www.youtube.com/watch?v=YVuolxil9Sw> (BuildinganLLM-PoweredText-BasedAIRPG)

<https://github.com/vossenwout/llm-rpg>

개발 환경 설명

HW 구성: 일반 PC

SW 구성:

언어: C++ (Object-Oriented Programming 기반)

그래픽/UI: 표준 콘솔 입출력

LLM 플랫폼: Ollama -> 기존 openapi 병용에서 ollama 단독으로 변경

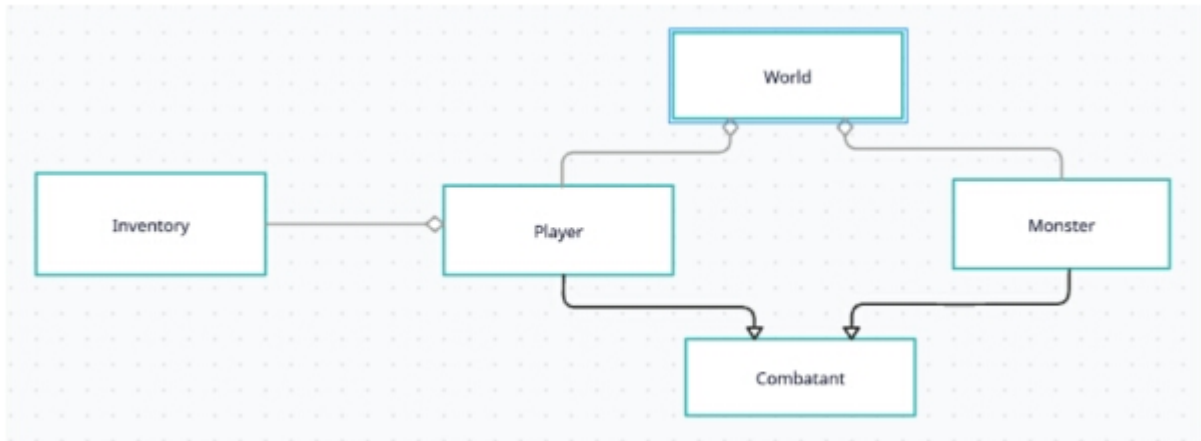
IDE: Visual Studio & Visual Studio Code

기능 설계: 플레이어 스탯 처리 → LLM 이벤트 생성 → 스토리 출력 → 사용자 피드백 순환 구조

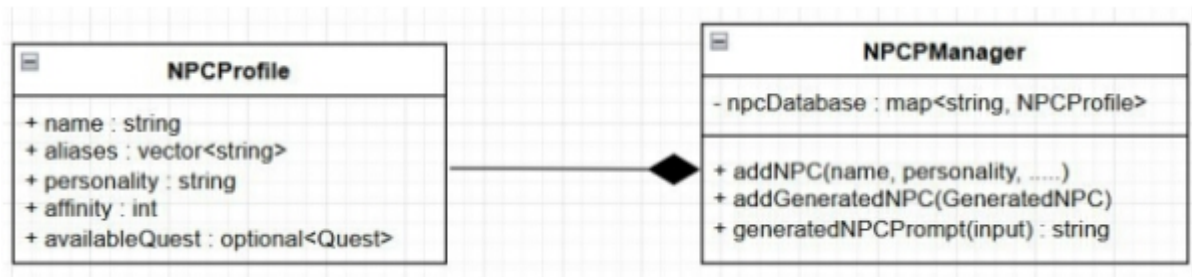
사용한 라이브러리 : openapi, ollama, sstream, algorithm, memory, optional, string, vector, exception, stdexcept, utility, map

+개발 프로그램 설명

-클래스 다이어그램 (파일 구성)



NPC



Item

src/llm/

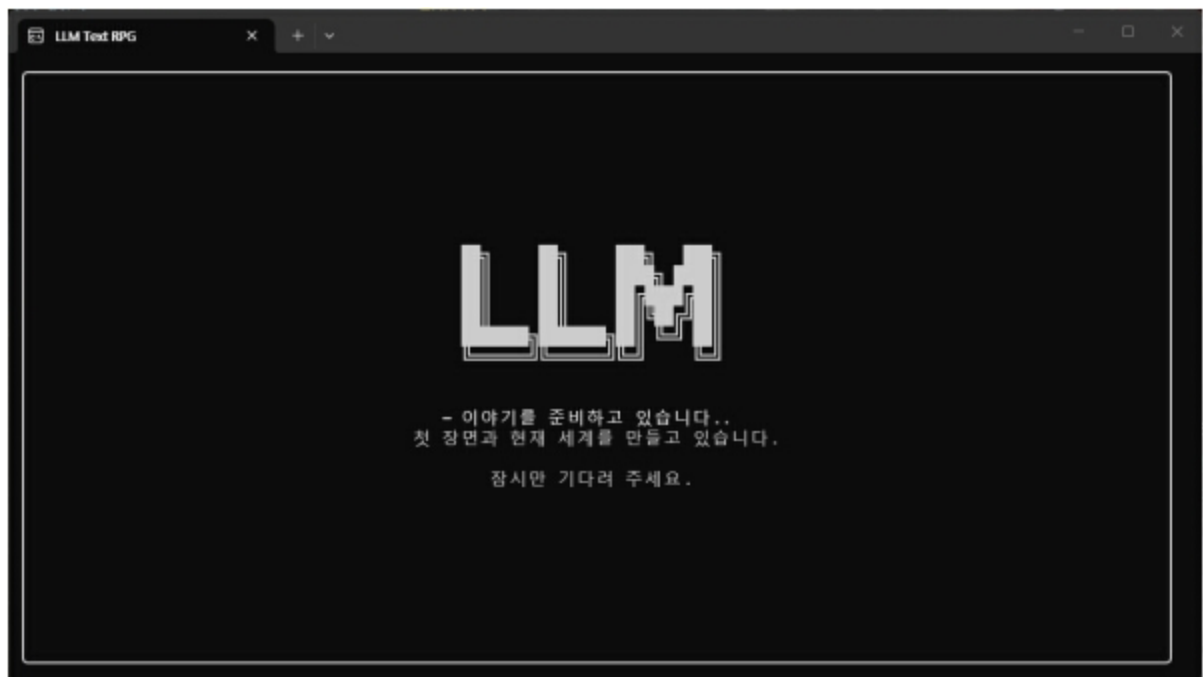
- |—— LLMTypes.hpp enum class ItemType, struct Item, struct PlayerSnapshot
(inventory)
- |—— ILLMClient.hpp <<abstract>> ILLMClient
- |—— OllamaClient.hpp/.cpp OllamaClient : public ILLMClient [final]
- |—— ContextBuilder.hpp/.cpp GameState → 컨텍스트 문자열 (인벤토리 표시)
- |—— PromptBuilder.hpp/.cpp 컨텍스트 → 완성 프롬프트
- |—— LLMOutputParser.hpp/.cpp LLM 원문 → GameEvent (parseItem 포함)

- └── LLMEventValidator.hpp/.cpp GameEvent 검증 + item value clamp
- └── LLMFallbackFactory.hpp/.cpp 실패 시 안전 GameEvent 생성
- └── LLMLogger.hpp/.cpp 파이프라인 로깅
- └── LLMService.hpp/.cpp 파이프라인 오케스트레이터 (진입점)
- └── LLMConfig.hpp/.cpp 설정 로더

-UI/UX 설계 (실제 화면 구조 이미지 및 사용자 경험과 인터페이스에 대한 설명)
인게임화면



로딩화면



모두 로딩 중에는 표시가 되어서 확인이 가능

클래스 다이어그램 (자세한 클래스 설명 및 핵심 기능 구현 상세
(주요 기능의 구현 방법과 주요 코드 설명)

Item 기능

<<abstract>>

ILLMClient

+ generate(prompt: string): string = 0

+ ~ILLMClient()



| 상속

OllamaClient [final]

- config_: OllamaConfig

+ generate(prompt: string): string [override]

LLMTypes.hpp (데이터 정의 전용 헤더)

enum class ItemType { Weapon, Armor, Consumable, QuestItem, Invalid }

enum class EventType { Story, Combat, ItemGain, ... , Invalid }

struct Item { name, type: ItemType, description, value: int }

struct Monster { name, description, hp, attack, defense }

struct StatChanges { hp, gold, exp }

struct PlayerSnapshot { hp, maxHp, level, attack, defense, gold, exp

inventory: vector<string> } ← 인벤토리

struct GameEvent { sceneText, eventType, choices,

item: optional<Item>, ← 아이템

monster: optional<Monster>,

statChanges, ... }

```
struct PromptSettings { maxInventoryItems = 8 }
```

```
struct ValidationConfig{ maxItemValue = 100 }
```

```
LLMOutputParser
```

```
+ parse(rawText: string): ParseResult
```

```
- extractJsonObject(...) [static]
```

```
- parseMonster(...) [static]
```

```
- parseItem(...) [static] ← JSON → optional<Item>
```

```
LLMEventValidator
```

```
- config_: ValidationConfig
```

```
+ validate(event: GameEvent): ValidationResult
```

```
  |—— item_gain이면 item 필수 검사
```

```
  |—— item.value → clamp(0, maxItemValue)
```

```
ContextBuilder
```

```
- settings_: PromptSettings
```

```
+ build(state: GameState, playerInput): string
```

```
  |—— inventory → appendLimitedList(maxInventoryItems)
```

```
LLMFallbackFactory
```

+ createSafeEvent(reason): GameEvent

└── item.reset() / monster.reset() ← 빈 상태로 초기화

LLMService (진입점)

- client_: shared_ptr<ILLMClient> ← 다형성

- contextBuilder_, promptBuilder_

- parser_, validator_

- fallbackFactory_, logger_

+ generateEvent(state, playerInput): GameEvent

ContextBuilder → PromptBuilder → ILLMClient::generate()

→ LLMOutputParser → LLMEventValidator → (실패 시 LLMFallbackFactory)

NPC

동적 NPC 확장성 (addGeneratedNPC)

코드에 미리 정의된 NPC뿐만 아니라, 게임 진행 중 LLM에 의해 새롭게 생성된 NPC 정보를 시스템에 동적으로 등록하고 관리할 수 있도록 설계함

```

void NPCManager::addGeneratedNPC(const textRpg::llm::GeneratedNPC& newNPC){
    if (npcDatabase.find(newNPC.name) == npcDatabase.end()){
        NPCProfile profile;
        profile.name = newNPC.name;
        profile.personality = newNPC.personality;
        profile.speechStyle = newNPC.speechStyle;
        profile.affinity = newNPC.affinity;
        profile.isMet = newNPC.isMet;

        npcDatabase[newNPC.name] = profile;
    }
}

```

Combat

Combatant 클래스 (데이터 및 기초 로직)

게임 내 전투를 수행하는 주체(플레이어, 몬스터)의 상태와 행동을 캡슐화한 클래스

주요 멤버 변수: name(이름), hp(현재 체력), maxHp(최대 체력), attack(공격력), defense(방어력/실드).

CombatSystem 클래스 (전투 제어 로직)

전투의 흐름(Turn기반)을 관리, 전투 결과 도출

run(): 전체 전투 과정을 실행하고 승패가 결정될 때까지 루프 돌림

Action(): 실제 공격 행위가 일어날 때 데미지를 계산하고 로그를 기록하는 세부 기능을 수행

주요 데이터 구조

SkillType: Attack, Defend 행동의 종류를 정의

CombatTurn: 각 턴마다 누가 어떤 행동을 했고, 데미지는 얼마였는지 등의 정보를 저장하는 로그 구조체

CombatResult: 전투 종료 후 승리자 정보와 모든 턴의 기록을 담아 반환하는 결과 객체

주요기능 설명

데미지 계산 함수

```

int receiveDamage(int damage)
{
    int resultDamage = 0;
    if (defense >= damage)
    {
        defense -= damage;
    }
    else
    {
        resultDamage = damage - defense;
        defense = 0;
        hp = std::max(0, hp - resultDamage);
    }
    return resultDamage;
}

```

std::max를 활용하여 체력이 음수가 되는 것을 방지

전투 진행 함수

```

CombatResult CombatSystem::run(Combatant player, Combatant monster) const
{
    CombatResult result;

    while (!player.isDead() && !monster.isDead())
    {
        // 플레이어 턴
        Action(CombatActor::Player, player, monster, SkillType::Attack, result);
        if (monster.isDead()) { break; }

        // 몬스터 턴
        Action(CombatActor::Monster, monster, player, SkillType::Attack, result);
    }

    //동시 사망시 플레이어 패배
    result.winner = (!player.isDead()) ? CombatWinner::Player : CombatWinner::Monster;

    result.player = std::move(player);
    result.monster = std::move(monster);
    return result;
}

```

개변수로 받은 player와 monster객체는 복사본이므로, 전투가 끝나도 원본데이터에는 영향을 주지 않고 전투결과(CombatResult)만 안전하게 반환

개발 중 발생한 장애요인과 문제 해결 과정

1. 중간 보고서에서 언급되었던 한국어 문제의 경우 ollama로 변경 이후엔 발생하지 않아 openapi와 관련된 문제였던 것으로 보임.

2. 유저들의 플레이 성향에 따라 게임이 매우 길어질 수 있는데 로컬 환경 LLM의 기억량 한계로 최종보스를 거점에서 만나는 장로가 특정하도록 하여 게임이 너무 길어지는것을 방지함.
3. 그럼에도 성능이 낮은 컴퓨터에서는 수십턴 이상 진행할 경우 LLM이 제대로 응답하지 않아 진행이 불가능한 경우가 있음.
4. UI의 공간적 제약과 아스키 아트로는 몰입감에 큰 도움이 되지 않을 것이라고 판단해 이미지를 구현하는 대신 사운드를 추가하였음
5. 전투 방식의 경우에 초기에는 덱빌딩 방식으로 확장하려고 하였으나 그렇게 할 경우 덱을 구현하고 이것을 텍스트로 하나하나 읽는 것이 플레이어의 스트레스를 유발할 것으로 판단해 최대한 간단하면서 직관적인 포켓몬스타일의 턴제 전투로 변경하여 4개의 스킬 및 5가지의 속성을 가지도록 변경함.

성과 및 결론

사용하는 컴퓨터의 사양에 맞는 LLM 버전을 고를 수 있도록 하여 자신의 사양에 맞게 진행할 수 있게 되었음. 전투에 스피드와 속성 등을 추가하여 좀 더 다채롭고 흥미진진한 전투가 가능하게 하였음.

사운드가 추가되어 승리, 아이템 획득 등 특정 상황에 피드백을 주어 몰입감을 향상시킴.

기존에는 게임 진행 화면이 거의 그대로 유지 되었으나 게임 시작 화면, 로딩 화면 등을 추가해 완성도를 향상시킴.

로컬 환경에서만 구동이 가능하도록 전투 시스템은 전부 c++ 코드 내에서 처리하게 하는 등 LLM이 가용 가능한 기억량을 최대한 사용하지 않도록 시도를 하였으나 그럼에도 사양에 따라서는 턴이 길어지면 진행이 불가능한 경우가 발생해 이럴때를 대비한 진행 방식이나 api를 다시 활용하는 기능을 추가해야할 것으로 보임.

기대효과 및 활용방안

AI가 플레이어의 스탯과 과거 선택들을 기반으로 하여 개인에게 적절한 난이도의 이벤트를 제공할 수 있게 될것입니다. 그로인해 특정 지역에서 오래 머물러서 다음 지역의 권장 레벨보다 과하게 강해져 게임의 흥미가 떨어지는 경우를 예방할 수 있을것으로 보입니다. 또한 플레이어의 성향에 따라 게임이 너무 길어지는 것을 막아 LLM의 기억량 보다 게임의 정보가 많아지는 것을 어느정도는 줄일 수 있을것으로 보입니다.

프로젝트 성과

LLM을 활용하여 과제나 코드를 작성하는데 그치지 않고 LLM이 게임의 직접적인 부분에 관여하게 함으로써 LLM을 더욱 깊이 이해하고 활용할 수 있게 된것같음. 또한 이 LLM에게 효과적으로 정보를 전달하기 위해 수업시간에 학습했던 여러 객체지향적 프로그래밍을 활용하여 클래스를 설

계하고 코드를 작성하는 능력을 기르게 됨. 또한 팀단위 프로젝트를 통해서 다른 팀원들의 코드를 이해하고 반대로 나의 코드를 팀원들이 쉽게 이해할 수 있도록 주석을 작성하거나 더 깔끔하고 직관적으로 코드를 작성할 필요를 느낌. 추가로 github에서 새로운 브랜치를 생성 혹은 병합하는 경험을 통해 conflict를 해결하는 등 git을 활용한 협업 프로젝트 경험을 얻을 수 있었음. 추가로 기존의 계획에서 더 확장되는 경우, 반대로 축소하거나 방향을 바꾸는 경우 등 장기 팀단위 프로젝트에서 목표를 변경하는 경험을 얻을 수 있었음.

실제 개발 일정

매주 금요일 오후 1시에 정기 회의를 가짐. 정기 회의에서는 그동안의 개발 성과를 공유하고 다시 다음주까지의 역할과 할당량을 정하였음. 그리고 그 외에는 각자 개인 시간을 할애하여 개발을 진행하였음. 특정 팀원의 사정으로 정기회의가 어려운 경우에는 그 인원을 제외한 인원끼리 회의 후 회의 내용을 참석하지 못한 인원에게 전달하거나 혹은 수업이 끝난 직후에 모여 대면으로 회의를 진행하였음.

역할 분담

- 안재영 : 전투 시스템 개선 / 발표자료 작성 / 발표
- 강준혁 : NPC 개발 / 플레이 테스트 및 녹화
- 김준형 : 아이템 개발 / 사운드 추가
- 차서훈 : LLM 전담 및 LLM - 게임 연결 개발 / 플레이 테스트 및 녹화 / 발표

객체지향 요소 표

요소	위치	내용
enum class	ItemType 등	Weapon,Armor,Consumable,QusetItem,Invalid-타입열거형
struct	Item 등	name,type,description,value-아이템데이터 묶음
std::optional	GameEvent::item	아이템 유무를 포인터 없이 타입으로 변환
static private메서드	LLMOutputParser::parseItem()	JSON→optional<Item>변환,외부 노출 차단
explicit생성자	LLMEventValidator(ValidationConfig)	밸런스수치입력,암묵적변환방지
추상화	generateNPCPrompt() 등	main에서 내부로직을 몰라도 정보 가져오기 가능
캡슐화	NPCProfile LLMEventValidator::config_	속성 그룹화 및 외부 수정 제한

	Combatant 등	
--	-------------	--

참고문헌

GeneratingNarrativeTextinaStoryWorld(논문)

<https://arxiv.org/abs/2308.07540>

EOF